



DESARROLLO DE SOFTWARE PARA PLATAFORMAS MOVILES

JONATHAN LÓPEZ LONDOÑO
JLOPEZL@UAO.EDU.CO
315 926 5443



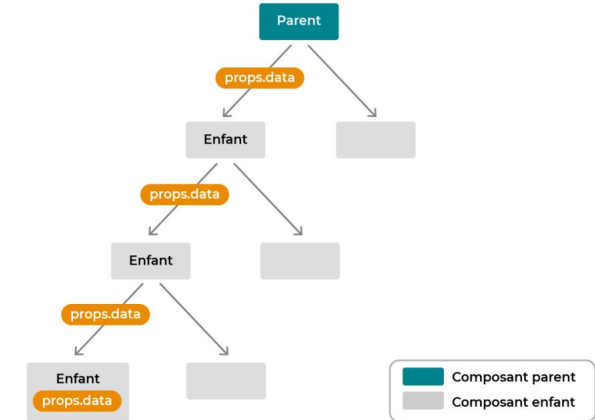
Context API

It is a way to share global state between multiple components without **prop drilling**.

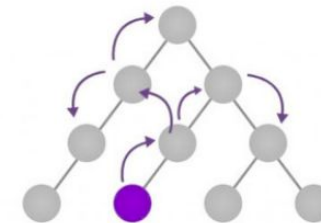
It is required to use one context and one provider.

A parent node will provide data and all its child nodes will consume that data. Therefore, we could create N contexts in React, one by parent who needs to share data

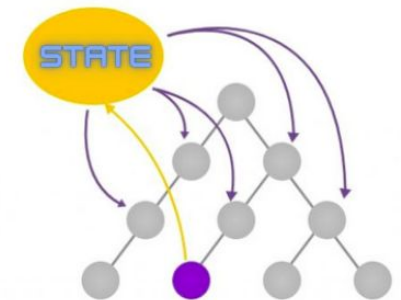
Prop drilling



without Context



with Context



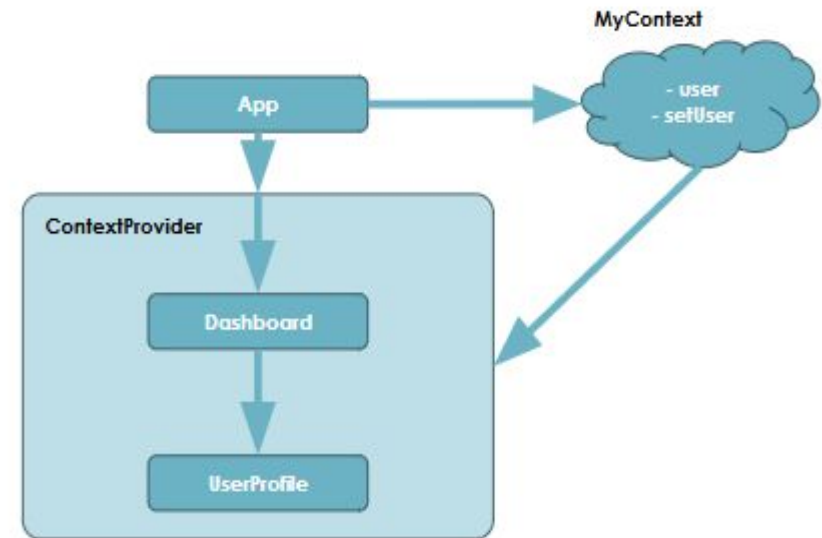
● Component initiating change

Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using `createContext`.
2. Use the Provider: The Provider is the component that sends data to all child components.
3. Wrap the components that will use it: All components that need to access the context must be inside the Provider.
4. Consume the Context: In any child component, the context can be accessed using `useContext`.



Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using **createContext**.



```
// src/context/MyContext.ts
import { createContext } from "react";

export const AuthContext = createContext();
```

Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using `createContext`.
2. Use the Provider: The **Provider** is the component that sends data to all child components.

In this example we are creating a context to share data/logic about the login, managing the `localStorage`

```
// src/context/MyContext.ts
import { createContext } from "react";

export const AuthContext = createContext();

export function AuthProvider({ children }) {

  const [user, setUser] = useState(null);

  useEffect(() => {
    const storedUser = localStorage.getItem("user");

    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = (userData) => {
    setUser(userData);
    localStorage.setItem("user", JSON.stringify(userData));
  };

  const logout = () => {
    setUser(null);
    localStorage.removeItem("user");
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```



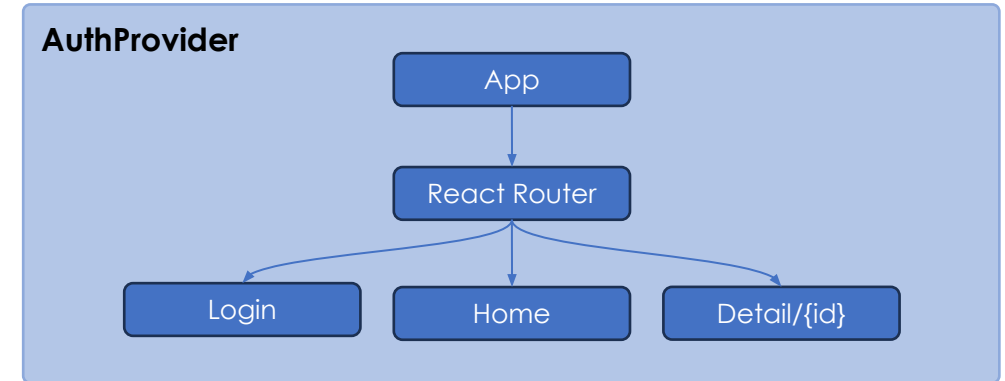
Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using `createContext`.
2. Use the Provider: The Provider is the component that sends data to all child components.
3. Wrap the components that will use it: All components that need to access the context must **be inside the Provider**.

In this example we are wrapping the whole app with the context, using the **AuthProvider**.



```
// src/main.tsx

import ReactDOM from 'react-dom/client'
import App from './App'
import { AuthProvider } from './context/AuthContext'

ReactDOM.createRoot(document.getElementById('root')).render(
  <AuthProvider>
    <App />
  </AuthProvider>
)
```

Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using `createContext`.
2. Use the Provider: The Provider is the component that sends data to all child components.
3. Wrap the components that will use it: All components that need to access the context must be inside the Provider.
4. Consume the Context: In any child component, the context can be accessed using **`useContext`**.

In this example, the Login page is using the **`AuthContext`** using the **`useContext`** hook to call the **login function from the Context**.

```
// src/pages/Login.tsx

import {
  IonPage,
  IonContent,
  IonInput,
  IonButton
} from "@ionic/react"

import { useState, useContext } from "react"
import { useHistory } from "react-router-dom"
import { AuthContext } from "../context/AuthContext"

function Login() {

  const [email, setEmail] = useState("")
  const history = useHistory()

  const { login } = useContext(AuthContext)

  const handleLogin = () => {
    const fakeUser = {
      name: "Jonathan",
      email: email
    }

    login(fakeUser)
    history.push("/dashboard")
  }

  return (
    <IonPage>
      <IonContent className="ion-padding">

        <h2>Login</h2>

        <IonInput
          placeholder="Email"
          onIonChange={(e) => setEmail(e.detail.value)}
        />

        <IonButton expand="block" onClick={handleLogin}>
          Ingresar
        </IonButton>
      </IonContent>
    </IonPage>
  )
}

export default Login
```

Creating the Context API

In React, Context allows to share data between multiple components without having to manually pass props at each level.

To create a context, follow this steps:

1. Create the Context: First, we create the context using `createContext`.
2. Use the Provider: The Provider is the component that sends data to all child components.
3. Wrap the components that will use it: All components that need to access the context must be inside the Provider.
4. Consume the Context: In any child component, the context can be accessed using **`useContext`**.

In this example, the Dashboard page is using the **`AuthContext`** using the **`useContext`** hook to call the user data and the logout function from the Context.

```
// src/pages/Dashboard.tsx

import {
  IonPage,
  IonContent,
  IonButton
} from "@ionic/react"

import { useContext } from "react"
import { useHistory } from "react-router-dom"
import { AuthContext } from "../context/AuthContext"

function Dashboard() {
  const { user, logout } = useContext(AuthContext)
  const history = useHistory()

  const handleLogout = () => {
    logout()
    history.push("/login")
  }

  return (
    <IonPage>
      <IonContent className="ion-padding">

        <h2>Dashboard</h2>

        <p>Bienvenido {user?.name}</p>

        <IonButton onClick={handleLogout}>
          Cerrar sesión
        </IonButton>

      </IonContent>
    </IonPage>
  )
}

export default Dashboard;
```

Custom Hooks

A function that reuses logic based on React hooks.

Main rule:

It must begin with the word "use"

Examples:

- **useAuth**
- useFetch
- useLocalStorage
- useUser
- useTheme

This tells React that the function uses hooks internally.

They are used to extract logic from the components.

```
// src/hooks/useAuth.tsx
export function useAuth() {

  const [user, setUser] = useState(null);

  useEffect(() => {
    const storedUser = localStorage.getItem("user");

    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = (userData) => {
    setUser(userData);
    localStorage.setItem("user", JSON.stringify(userData));
  };

  const logout = () => {
    setUser(null);
    localStorage.removeItem("user");
  };

  return {user, login, logout}
}
```

Custom Hooks

This example is a component without Custom Hooks

```
// src/context/MyContext.ts
import { createContext } from "react";

export const AuthContext = createContext();

export function AuthProvider({ children }) {

  const [user, setUser] = useState(null);

  useEffect(() => {
    const storedUser = localStorage.getItem("user");

    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = (userData) => {
    setUser(userData);
    localStorage.setItem("user", JSON.stringify(userData));
  };

  const logout = () => {
    setUser(null);
    localStorage.removeItem("user");
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Custom Hooks

This example is the same component using Custom Hooks.

Now the logic belongs to a custom Hook and the component is as clean as possible.



```
// src/context/MyContext.ts
import { createContext } from "react";
import { useAuth } from "../hooks/useAuth.tsx";

export const AuthContext = createContext();

export function AuthProvider({ children }) {

  const { user, login, logout } = useAuth();

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```



```
// src/hooks/useAuth.tsx
export function useAuth() {

  const [user, setUser] = useState(null);

  useEffect(() => {
    const storedUser = localStorage.getItem("user");

    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = (userData) => {
    setUser(userData);
    localStorage.setItem("user", JSON.stringify(userData));
  };

  const logout = () => {
    setUser(null);
    localStorage.removeItem("user");
  };

  return {user, login, logout}
}
```

Firebase

Firebase is a platform that gives a ready-to-use backend as a service (BaaS).

It includes:

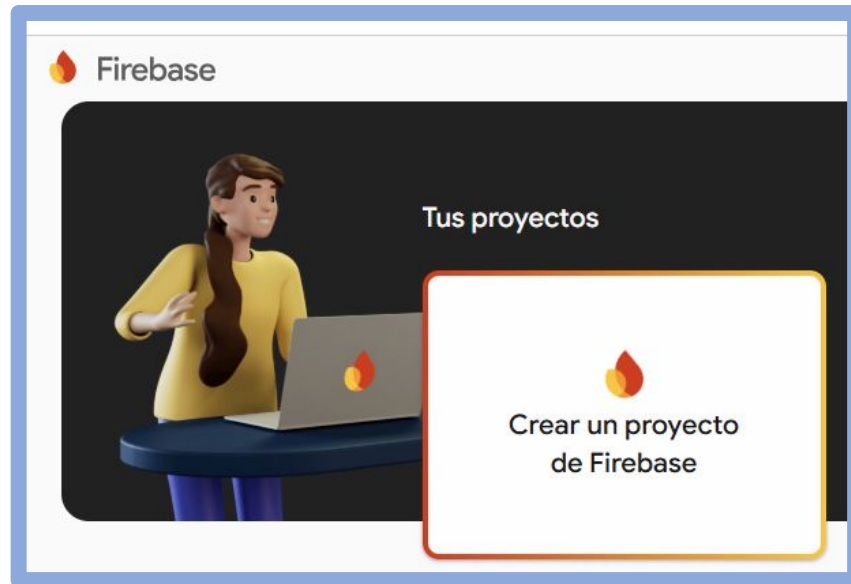
- ✓ Authentication
- ✓ Real-time database
- ✓ File storage
- ✓ Hosting
- ✓ Push notifications
- ✓ Analytics
- ✓ All connected and ready to use from the frontend.



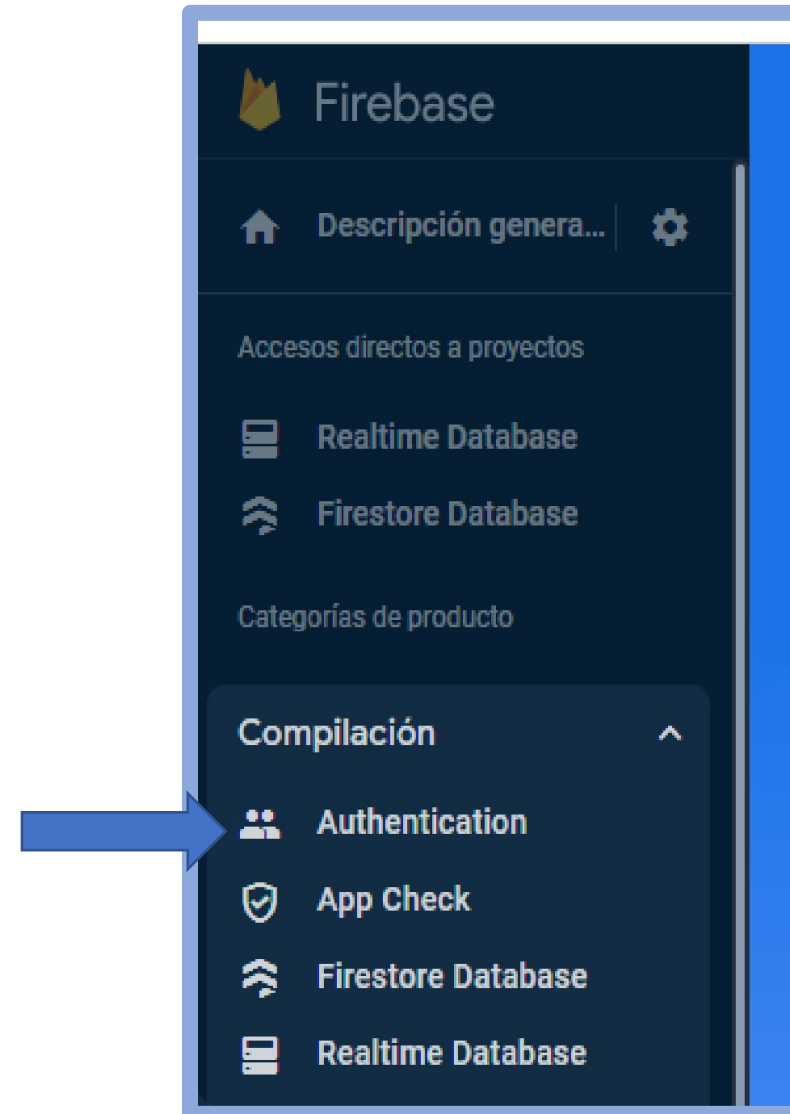
Firebase Login

<https://console.firebase.google.com/>

1. Create a new project



2. Once the Project has been created, go to Authentication in the left menu

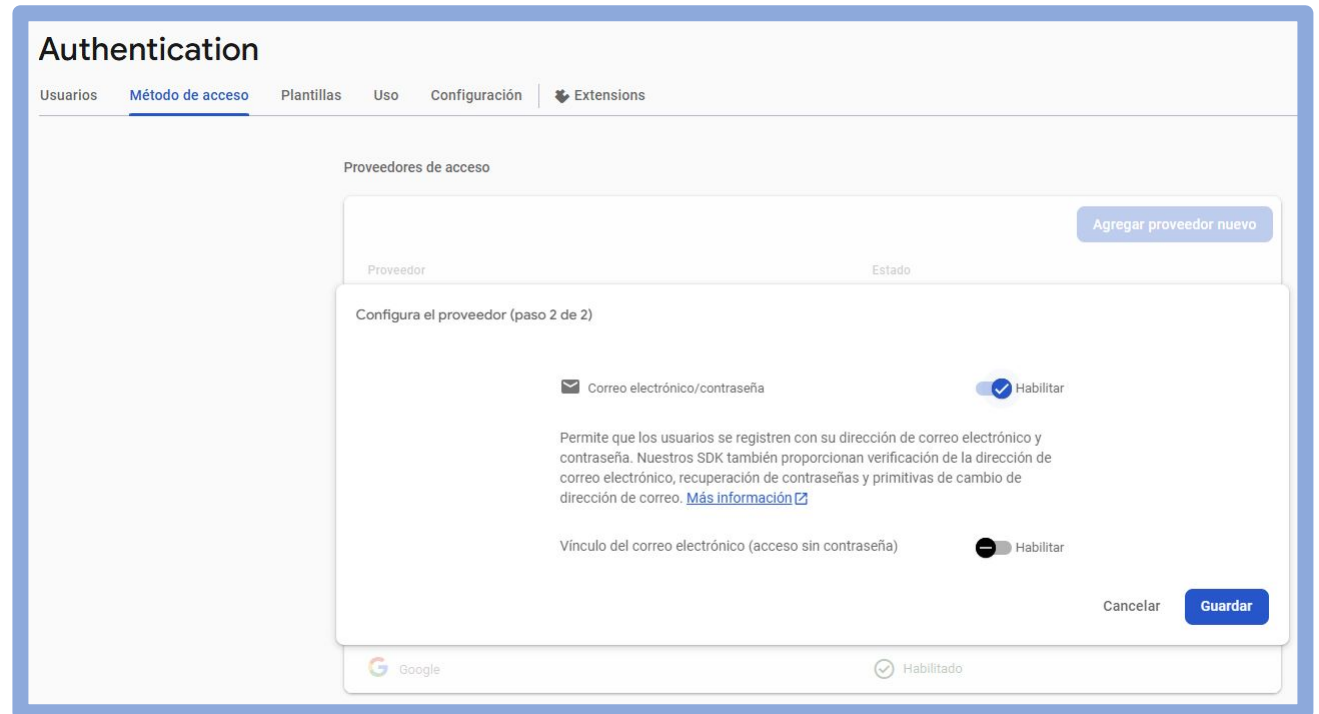
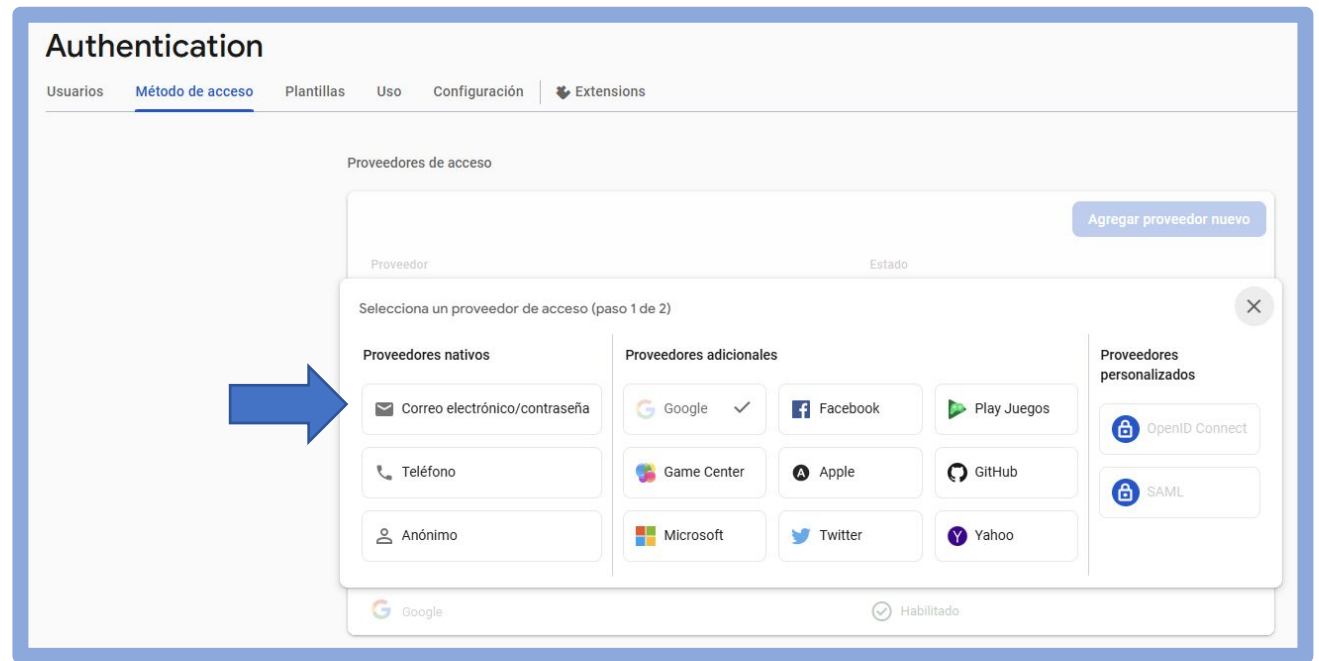


Firebase Login

<https://console.firebase.google.com/>

3. Choose email/password, you can also choose other options.

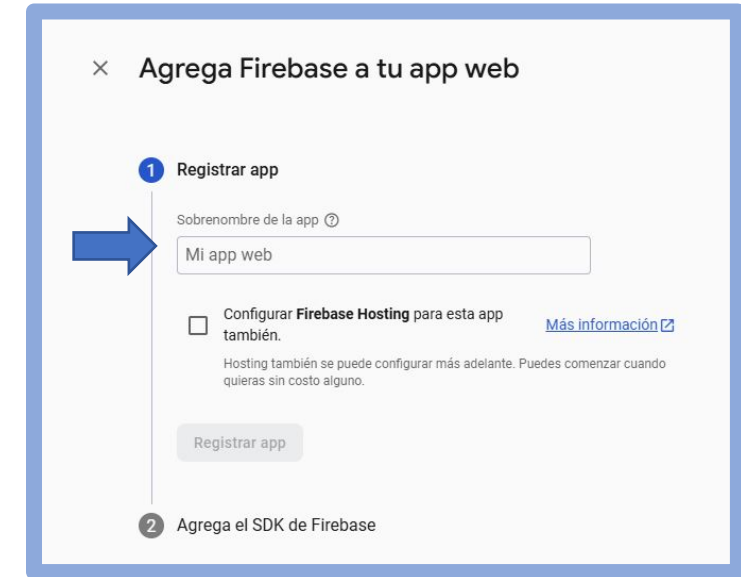
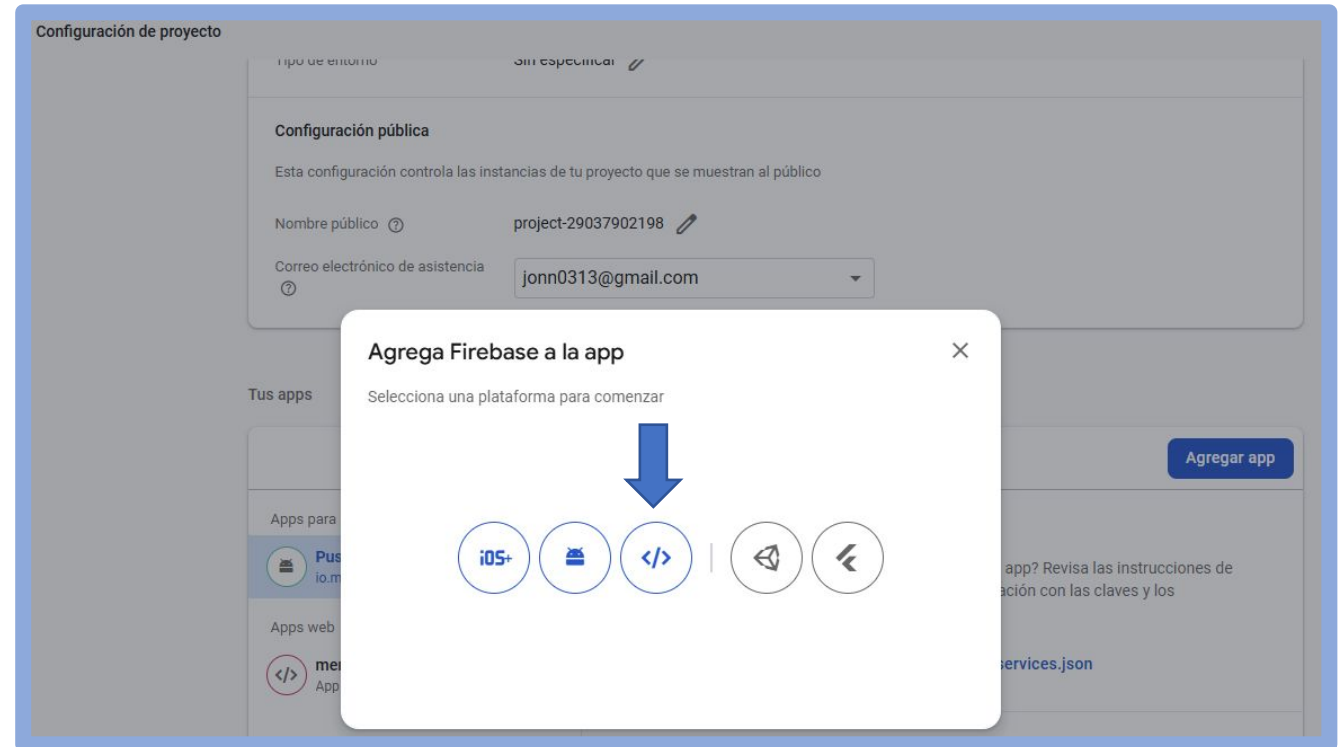
Then Firebase authentication is already created



Firestore Login

<https://console.firebase.google.com/>

4. Go to the left menu firebase and add a new app in the General Description option



Firestore Login

<https://console.firebase.google.com/>

5. Copy the resultant code.



2 Agrega el SDK de Firebase

☒ Usar npm ☐ Usar una etiqueta <script>

Si ya usas [npm](#) y un agrupador de módulos como [Webpack](#) o [Rollup](#), puedes ejecutar el siguiente comando para instalar la versión más reciente del SDK ([más información](#)):

```
$ npm install firebase
```

Luego, inicializa Firebase y comienza a usar los SDK de los productos que quieres utilizar.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyAaMrEwa9rlbjNusXZW1MUakH5OYkRpFg",
  authDomain: "mente-360.firebaseio.com",
  databaseURL: "https://mente-360-default-rtdb.firebaseio.com",
  projectId: "mente-360",
  storageBucket: "mente-360.firebaseio.com",
  messagingSenderId: "29037902198",
  appId: "1:29037902198:web:ae8e99959212d14c766257"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

Nota: Esta opción utiliza el [SDK de JavaScript modular](#), que proporciona un tamaño reducido del SDK.

Obtén más información sobre Firebase para la Web: [primeros pasos](#), [referencia de la API del SDK web](#) y [muestras](#)

Now, it's time to go back to our Ionic project...

1. Install firebase library: **npm i firebase**
2. Create a new folder called firebase into SRC
3. Create a new file into firebase folder called config.js
4. Paste the firebase config in this file.

In this example we are using Firebase authentication.
That's why we need to add this lines to the file:

```
import {getauth } from 'firebase/  
...  
const auth = getauth()
```

```
import { getAuth } from 'firebase/auth'  
import { initializeApp } from "firebase/app";  
  
const firebaseConfig = {  
  apiKey: "AIzaSyAlz5wyoYl_6iZCS1yzBVYP4cejL0e5LaA",  
  authDomain: "vue-demo-1c3f5.firebaseio.com",  
  databaseURL: "https://vue-demo-1c3f5-default-rtdb.firebaseio.com",  
  projectId: "vue-demo-1c3f5",  
  storageBucket: "vue-demo-1c3f5.appspot.com",  
  messagingSenderId: "318224169700",  
  appId: "1:318224169700:web:51362c4bf539f5487c0af1"  
};  
  
// Initialize Firebase  
const app = initializeApp(firebaseConfig);  
  
//initialize firebase auth  
const auth = getAuth()  
  
export { app, auth }
```

Firestore Login

Create and use a new Login page.

This component uses **signInWithEmailAndPassword**, a firebase method to login with email and password.

```
// src/pages/Login.tsx

import {
  IonPage, IonContent, IonInput, IonButton
} from "@ionic/react"

import { useState } from "react"
import { signInWithEmailAndPassword } from "firebase/auth"
import { auth } from "../firebase"
import { useHistory } from "react-router-dom"

function Login(){

  const [email,setEmail] = useState("")
  const [password,setPassword] = useState("")

  const history = useHistory()

  const handleLogin = async () => {
    try{
      await signInWithEmailAndPassword(
        auth, email, password
      )
      history.push("/dashboard")
    }catch(error){
      alert(error.message)
    }
  }

  return(
    <IonPage>
      <IonContent className="ion-padding">

        <h2>Login</h2>

        <IonInput
          placeholder="Email"
          onChange={(e)=>setEmail(e.detail.value)}
        />

        <IonInput
          type="password"
          placeholder="Password"
          onChange={(e)=>setPassword(e.detail.value)}
        />

        <IonButton expand="block" onClick={handleLogin}>
          Login
        </IonButton>

      </IonContent>
    </IonPage>
  )
}

export default Login
```

Firestore Login

Create and use a new Register page.

This component uses **createUserWithEmailAndPassword**, a firebase method to create users just with email and password.



```
// src/pages/Register.tsx

import {
  IonPage, IonContent, IonInput, IonButton
} from "@ionic/react"

import { useState } from "react"
import { createUserWithEmailAndPassword } from "firebase/auth"
import { auth } from "../firebase"
import { useHistory } from "react-router-dom"

function Register(){

  const [email,setEmail] = useState("")
  const [password,setPassword] = useState("")

  const history = useHistory()

  const handleRegister = async () => {
    try{
      await createUserWithEmailAndPassword(
        auth,
        email,
        password
      )

      history.push("/dashboard")
    }catch(error){
      alert(error.message)
    }
  }

  return(
    <IonPage>
      <IonContent className="ion-padding">

        <h2>Registro</h2>

        <IonInput
          placeholder="Email"
          onChange={(e)=>setEmail(e.detail.value)}
        />

        <IonInput
          type="password"
          placeholder="Password"
          onChange={(e)=>setPassword(e.detail.value)}
        />

        <IonButton expand="block" onClick={handleRegister}>
          Crear cuenta
        </IonButton>
      </IonContent>
    </IonPage>
  )
}

export default Register
```

Challenge 05

Transform the Tasks App to add firebase Login/Register, custom hooks and contexts.

- ✓ The app must contain:
 - ✓ Login – Register – Logout with firebase
 - ✓ routing pages: Login, Register, Tasks list, Add / Edit Task, Task Detail
- ✓ Use one context to handle Tasks data through pages.
- ✓ Use one different context to handle login - Register data.
- ✓ Use custom hooks to handle Firebase logic

